Algoritmen en Datastructuren II
Taak 2 - SAT

Academiejaar 2025-2026 (1e zittijd)
Assistenten: Carlo Emerencia (Carlo.Emerencia@vub.be)
Doryan Temmerman (Doryan.Temmerman@vub.be)

1 Inleiding

Een logische veralgemening van het SAT probleem is het MaxSAT probleem, zoals we in de cursus zagen. In deze taak zal je een MaxSAT-solver implementeren. We gebruiken hiervoor hetzelfde algortime als in de cursus: LSU. Deze staat al high-level geïmplementeerd in de file `maxsat.rkt`, maar de hulpprocedures ontbreken nog. Je zal hierbij zelf ook nog het `CNF` ADT moeten aanpassen om met MaxSAT formules om te kunnen, en de ontbrekende hulpprocedures moeten implementeren.

2 Opgave

Het doel van deze taak is het **implementeren van het LSU-algoritme**. Dit doe je in het bestand `maxsat.rkt` in de bijgeleverde folder `a-d/sat`. Dit bestand bevat een basis-skelet die je dient te vervolledigen. Het bestand `main.rkt` stelt je in staat om je implementatie te demonstreren en te testen.

2.1 Dimacs voor MaxSAT

We zullen MaxSAT formules inlezen van aangepaste dimacs files. Doordat bepaalde clauses gewichten hebben moet de format lichtjes verschillen van de dimacs files voor SAT. Je kan meer uitleg vinden over dit gestandardiseerd format op deze link. Je kan een aangepaste dimacs parser vinden in de bijgeleverde bestanden.

Het `CNF` ADT zal verder moeten aangepast worden om met gewogen clauses om te gaan.

2.1.1 Opdracht:

- Pas het `CNF` ADT uit `cnf.rkt` aan zodat deze ook met gewogen clauses om kan. De procedure `make-clause` gebruikt het signatuur `list<literal> number weight`

`-> clause`, waarbij `weight` ofwel `#f` ofwel een `number` is.

- Pas ook de procedure `make-formula` aan zodat deze het signatuur `list<clause> list<clause> number number -> formula` heeft. De twee lijsten stellen de hard-clauses en de soft-clauses voor.

2.2 Cost function

Het LSU algoritme maakt op een essentiële manier gebruik van de *cost* function. Deze wordt namelijk in elke iteratie opgeroepen om de huidige beste kost te berekenen.

2.2.1 Opdracht

- Schrijf de procedure `cost` die, gegeven een formule en een interpretatie, de kost hiervan uitrekent.

2.3 Blocking Variable Transform

Als pre-processing bij het LSU-algoritme wordt een Blocking Variable Transformation toegepast. Herinner dat hierbij nieuwe blocking/relaxing variabelen worden aangemaakt die zullen instaan voor de soft clauses van de formule.

2.3.1 Opdracht

- Schrijf de procedure `blocking-variable-transform!` die, gegeven een formule, deze formula destructief aanpast door het Blocking Variable Transformation toe te passen.

2.4 Pseudo-Boolean constraint encoding

Om de formule $\mathcal{D}(\mathcal{F}, k)$ op te stellen zullen we geen gebruik maken van Binary Decision Diagrams zoals in de cursus, maar van *Generalized Totalizer Encoding* (GTE). Deze maakt ook gebruik van een binaire boom om de informatie weer te geven, maar op een andere manier dan BDDs. We zullen opnieuw de pseudo-Boolean constraint

$$2\ell_1 + 3\ell_2 + 5\ell_3 \leq 6,$$

als leading voorbeeld nemen.

Dit process staat reeds in de meegegeven procedure `pseudo-boolean-layer` in de file `maxsat.rkt`.

2.4.1 Constructie van de GTE boom

De GTE-boom is een (niet noodzakelijk complete) binaire boom waarvan alle nodes een lijst zullen bevatten van tupels van variabelen met een geassociëerd gewicht. De lijsten

van de leaves bevatten elk exact één tupel: een literal uit de pseudo-Boolean constraint en diens overeenkomstige gewicht.

De volledige boom wordt bottom-up opgebouwd. Alle layers worden in iteraties geconstrueerd door twee nodes van de layer eronder samen te verbinden via een parent-node. Deze parent-node stelt alle mogelijke waarden voor van gewogen sommen van de literals die zich dieper in de boom onder deze node bevinden. Hiervoor worden er per nieuwe node meerdere nieuwe variabelen aangemaakt, één per mogelijke waarde van de som, en deze wordt aan de lijst van de node toegevoegd in een tupel. Een som van 0 wordt nooit toegevoegd.

Voorbeeld

We bekijken het voorbeeld $2\ell_1 + 3\ell_2 + 5\ell_3 \leq 6$, zie Figuur 1. Voor elke literal wordt een node aangemaakt.

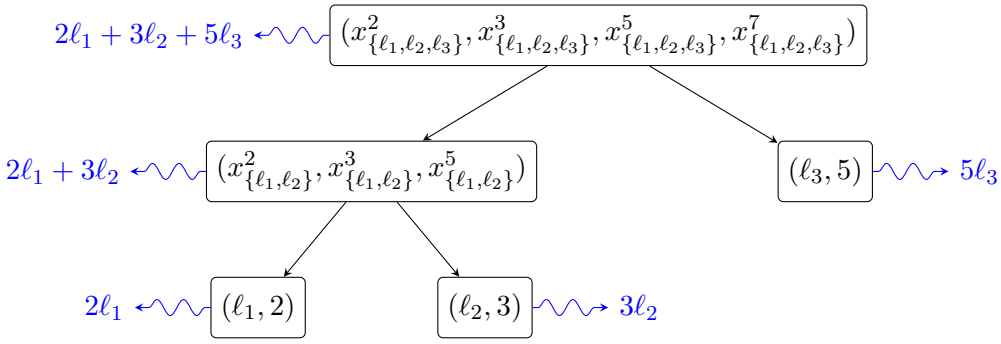
In de eerste iteratie wordt boven de leaves $(\ell_1, 2)$ en $(\ell_2, 3)$, die respectievelijk de literals ℓ_1 en ℓ_2 voorstellen, een nieuwe node aangemaakt. Deze stelt alle mogelijke waarden van $2\ell_1 + 3\ell_2$ voor. Dit zijn dus $0, 2, 3$ en 5. Voor de waarde 0 wordt geen nieuwe variabele aangemaakt, dus dit levert 3 nieuwe variabelen op: $x^2_{\{\ell_1,\ell_2\}}, x^3_{\{\ell_1,\ell_2\}}$ en $x^5_{\{\ell_1,\ell_2\}}$. De lijst voor deze nieuwe node is dan $(x^2_{\{\ell_1,\ell_2\}}, x^3_{\{\ell_1,\ell_2\}}, x^5_{\{\ell_1,\ell_2\}})$.

Merk op dat de leaf voor ℓ_3 geen partner heeft om samen te verbinden tot een nieuwe node (alle leaves zijn opgebruikt). We zullen in dat geval een iteratie wachten tot deze wel een partner vindt.

Tijdens de volgende iteratie kunnen we de node die $2\ell_1 + 3\ell_2$ voorstelt samenvoegen met de node voor ℓ_3. Alle mogelijke waarden zijn nu $0, 2, 3, 5, 7, 8$ en 10. We zouden nu à priori 6 variabelen moeten toevoegen aan onze nieuwe node. Het blijk echter dat we met 4 variabelen voldoende zullen hebben. Inderdaad, ons doel is om $2\ell_1 + 3\ell_2 + 5\ell_3 \leq 6$ te encoderen, dus we zijn eigenlijk enkel geïnteresseerd in de exacte waarden van de sommen als ze kleiner blijven dan 6. Van zodra een som 7 of meer wordt zal ze verder 7 of meer blijven (doordat de gewichten positief zijn). We voegen dus voor de gewichten $7, 8$ en 10 slechts één variabele toe, en geven deze het gewicht 7. Dit heet *trimming*. De node krijgt dus het label $(x^2_{\{\ell_1,\ell_2,\ell_3\}}, x^3_{\{\ell_1,\ell_2,\ell_3\}}, x^5_{\{\ell_1,\ell_2,\ell_3\}}, x^7_{\{\ell_1,\ell_2,\ell_3\}})$. Merk op dat ook maar één variabele met het gewicht 5 wordt toegevoegd, hoewel 5 op meerdere manieren kan bereikt worden.

Na deze iteratie is de boom klaar. In Figuur 1 zijn de ook de sommen die elke node voorstelt bijgevoegd in het blauw.

Zoals je hierboven kan opmerken, gebruiken we de volgende afspraak voor het benoemen van de variabelen: voor een node die de som voorstelt van de verzameling van literals L voegen we de variabele x^w_L toe als w een mogelijk gewicht (rekening houdend met trimming) voor die som is. In het bijzonder is de w dus geen macht, maar een index. Het wordt door deze conventie ook mogelijk om op zicht te zien tot welke node een bepaalde variabele behoort.



Figuur 1: De GTE-boom die $2\ell_1 + 3\ell_2 + 5\ell_3 \leq 6$ voorstelt.

2.4.2 Constructie van $\mathcal{D}(\mathcal{F}, k)$

In elke node (buiten de leaves) worden nieuwe variabelen aangemaakt die overeenkomen met een gewicht. De betekenis die we zullen geven aan zo'n variabele x_L^w (met $w \leq k$) is:

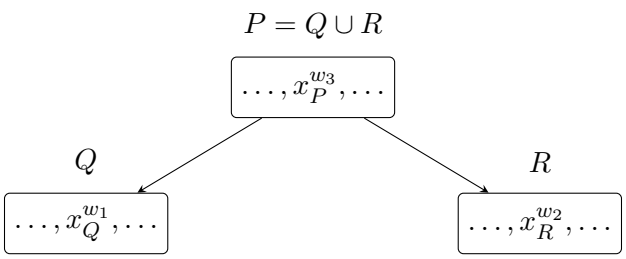
x_L^w moet **t** zijn wanneer een deelverzameling van literals van L, die zelf waar zijn, als som exact w heeft.

Doordat we aan trimming doen, is de betekenis voor een variabele met gewicht $k+1$ lichtjes anders:

x_L^{k+1} moet **t** zijn wanneer een deelverzameling van literals van L, die zelf waar zijn, als som minstens $k+1$ heeft.

Dit zal ons helpen om, zoals in het BDD geval, clauses op te stellen die aan de $\mathcal{D}(\mathcal{F}, k)$ moeten toegevoegd worden voor elke parent node met z'n twee children.

Veronderstel dat we een parent node in onze boom gekozen hebben. Elk van zijn twee kinderen komt overeen met een verzameling aan literals: bijvoorbeeld Q en R. De verzameling van literals die de parent node voorstelt is dan $P = Q \cup R$. Lokaal zitten we dus in de volgende situatie:



Als de variabele $x_Q^{w_1}$ waar is, dan bestaan er literals uit Q, die waar zijn, waarvan de gewogen som w_1 is, en als $x_R^{w_2}$ waar is, dan bestaan er literals uit R, die waar zijn, waarvan de gewogen som w_2 is. In dat geval bestaan er dus literals uit P, die waar zijn,

waarvoor de som $w_1 + w_2$ is! Door onze constructie zal er dus een variabele $x_P^{w_3}$ zijn in P waarvoor $w_3 = w_1 + w_2$. Deze variabele zal nu ook waar moeten zijn. We voegen dus aan $\mathcal{D}(\mathcal{F}, k)$ de clause

$$(x_Q^{w_1} \wedge x_R^{w_2}) \rightarrow x_P^{w_3} \equiv \overline{x}_Q^{w_1} \vee \overline{x}_R^{w_2} \vee x_P^{w_3},$$

toe wanneer $w_3 = w_1 + w_2$[1].

Op zichzelf geldt natuurlijk ook dat als de variabele $x_Q^{w_1}$ waar is, dat er dan literals uit $P \supset Q$ ook een som van w_1 hebben. Hierdoor voegen we ook de clause

$$x_Q^{w_1} \rightarrow x_P^{w_1} \equiv \overline{x}_Q^{w_1} \vee x_P^{w_1},$$

toe. Analoog voegen we

$$x_R^{w_1} \rightarrow x_P^{w_1} \equiv \overline{x}_R^{w_1} \vee x_P^{w_1},$$

toe.

De formule $\mathcal{D}(\mathcal{F}, k)$ moet natuurlijk uitdrukken dat de som van de gewichten van alle literals die **t** zijn niet strict groter is dan k. Deze som wordt voorgesteld door de root van de boom, en doordat we aan trimming gedaan hebben is er exact één variabele in deze root die een gewicht strict groter dan k heeft (nl. gewicht $k+1$). Dit is de variabele x_L^{k+1} (waarbij L de volledige verzameling van alle literals is). We voegen nu $\overline{x}_L^{k+1}$ als laatste clause toe aan de formule om uit te drukken dat de som niet strict groter is dan k.

2.4.3 De procedure `pseudo-boolean-layer`

De procedure `pseudo-boolean-layer` heeft als input een formule $\mathcal{F}$ en het getal k. We veronderstellen dat op de inputformule al de Blocking Variable Transform is toegepast. Deze procedure geeft de pseudo-Boolean layer (formule $\mathcal{F}_k$) terug.

De sub-procedure `make-tree` stelt de volledige GTE-boom op, met node 0 als root. Hiervoor wordt een gelabelde boom gebruikt. Om de diepte en het aantal nodes te berekenen van deze boom zijn enkele hulpprocedures gedefiniëerd.

De effectieve labels en het opstellen en toevoegen van de formule $\mathcal{D}(\mathcal{F}, k)$ gebeurt achteraf via `DFT` in `add-clauses-from-tree!`. Voor de uitleg gebruiken we P, Q en R zoals hierboven. De tupels zullen als `cons`-cellen geïmplementeerd worden.

Labels van een parent-node P worden in de procedure van `edge-processed` toegevoegd. De eerste keer dat er een edge processed wordt met `from`-knoop P zal de edge `(P . Q)` zijn. Dit kunnen we herkennen doordat P op dat moment geen label heeft. Op dit moment wordt tijdelijk het label van Q gekopiëerd naar P.

Wanneer de tweede edge met `from`-knoop P geprocessed wordt (de edge `(P . R)`) zal P al een tijdelijk label hebben. Nu wordt het definitieve label van P opgesteld: er worden nieuwe variabelen aangemaakt voor elke mogelijke waarde (gewicht) dat de node P zal kunnen hebben. Voor alle gewichten uit Q moet dit sowieso gebeuren. Daarna

[1] Dit klopt niet volledig: gezien we alle gewichten trimmen tot $k+1$ moet dit eigenlijk $w_3 = Min(w_1 + w_2, k+1)$ zijn.

moeten ook alle gewichten uit R (die niet al zijn aangemaakt) en en combinaties van gewichten met Q toegevoegd worden.

Op elk moment dat we een variabele uit Q, R of de som van zo'n twee controleren voegen we ook de overeenkomstige clauses toe aan $\mathcal{F}_k$.

Wanneer een node wordt geprocessed die geen label heeft, zal deze node een leaf moeten zijn door de vorige uitleg. We gebruiken dit in de procedure `node-processed` om het label van een leaf aan te maken.

2.4.4 Opdracht

- De procedure `pseudo-boolean-layer` is zeer inefficiënt opgesteld. Vervang de code zodat deze nog steeds hetzelfde gebruik (met signatuur `formula number -> formula`) behoudt, maar zodat deze geen boom aanmaakt in het geheugen.

3 Testen

In de bijgeleverde folder `a-d/sat/examples` zitten 6 voorbeelden van MaxSAT formules die je kan gebruiken om te testen. De optimale oplossing staat bij elke formule in de comments. Indien een formule te lang duurt om uit te werken via het algoritme, overweeg om de verbositeit van de logger (`logger.rkt`) aan te passen.

Indien je zelf andere voorbeelden wilt toevoegen (in dimacs formaat), kan je deze hier plaatsen.

4 Beoordeling

- **Deadline**: Upload je oplossing naar Canvas ten laatste op **zondag 24 mei 2026 23u59** in de daarvoor voorziene ruimte in de 'Opdrachten' rubriek.

- Je oplossing zit in een zip-mapje met je naam en studentennummer, bvb. `Doryan-Temmerman-123456.zip`. Deze map bevat de volgende bestanden:
  1. Een README bestand **(in .pdf)** waarin je een beschrijving geeft van je oplossing (maximaal 2 pagina's). Je beschrijft en motiveert de keuzes die je gemaakt hebt om tot je oplossing te komen. Je licht ook kort toe welke bestanden van de `a-d` folder je gebruikt en/of gewijzigd hebt.
  2. De `a-d` folder die de nodige ADTs bevat. ADTs die niet nodig zijn voor deze taak laat je weg uit deze folder om de grootte van je inzending te beperken. Zorg er wel voor dat alle nodige ADTs om je code te runnen aanwezig zijn! Dit kan je best apart testen alvorens je code op te sturen.
  3. De folder `a-d/sat/examples` waarin alle `.wcnf`-bestanden die MaxSAT problemen voorstellen verzameld zijn.
  4. Het bestand `maxsat.rkt`, eveneens in de `a-d/sat/` folder, waarin het algoritme wordt geïmplementeerd volgens de instructies hierboven.

5. Het bestand `main.rkt` waarin het algoritme wordt toegepast op alle voorbeeld formules.

- Merk op dat je code ook getest zal worden op andere MaxSAT problemen.

Je code wordt niet alleen beoordeeld op **functionaliteit** en **performantie** maar ook op **programmeerstijl** (abstracties, encapsulatie, etc.).

5 Opmerkingen

- De deadline is strikt na te leven: taken die te laat worden ingediend, zullen niet verbeterd worden en als afwezig worden gequoteerd.
- Indien je vragen hebt over de taak, neem je **tijdig** contact op met de assistent.

5.1 Wat is (niet) toegelaten?

De taak dient **strikt individueel** en op eigen kracht gemaakt te worden. Dat betekent dat je jouw taak op een zelfstandige basis moet maken en je jouw werk moet kunnen uitleggen, onder toezicht moet kunnen herimplementeren, en je werkwijze moet kunnen verdedigen. Werk (code, tekst, etc.) overnemen van of delen met derden (bv. medestudenten, websites, GitHub, etc.) is verboden. Merk op: het is dus ook expliciet verboden om werk gegenereerd door AI-tools (bv. gebaseerd op "large language models") in te dienen als eigen werk. **Over het algemeen is het gebruik van (generatieve) AI-tools voor dit project niet toegelaten, met uitzondering van grammatica-checkers voor de documentatie. Wees hier dan wel transparant over: je dient te vermelden welke tool(s) je hebt gebruikt.** Elektronische hulpmiddelen worden gebruikt om alle inzendingen met online bronnen en met elkaar te vergelijken, zelfs over verschillende academiejaren heen.

- De enige code die mag overgenomen worden, is de code gegeven in deze cursus: zowel de gegeven cursuscode op Canvas als de oplossingen van de WPOs van dit vak. Verder mag je enkel gebruikmaken van de bibliotheken die deel uitmaken van de R7RS-standaard.
- **Indien je oplossingen van het WPO hergebruikt of aanpast, vermeld je het respectievelijke WPO als bron in de README.**

5.2 Wat is plagiaat?

Elke handeling van een student die afwijkt van de gegeven instructies en niet in overeenstemming is met het examenreglement wordt beschouwd als onregelmatigheid. Plagiaat is eveneens een onregelmatigheid. Onder plagiaat wordt begrepen het gebruik maken van andermans werk, al dan niet in bewerkte vorm, zonder nauwkeurige bronvermelding

(cf. OER[2], Artikel 118§2). Plagiaat kan betrekking hebben op verschillende vormen van werk zoals tekst, code, beeld, etc. Ook de uitvoer van generatieve AI-tools overnemen zonder bronvermelding hieromtrent is plagiaat.

5.3 Wat gebeurt er als studenten verdacht worden?

Elk vermoeden van plagiaat zal onverwijld aan de decaan van de faculteit worden gerapporteerd. Zowel gebruiker als verlener van zulke code zullen worden gerapporteerd en zullen worden behandeld volgens de plagiaatregels van het examenreglement (cf. OER, Artikel 118). De decaan kan beslissen tot (een combinatie van) examentuchtsancties, gaande van 0 punten op het werkstuk tot een verbod tot (her)inschrijving voor één of meerdere academiejaren (cf. OER, Artikel 118§5).

Contacteer de assistent(en) als je twijfelt of iets al dan niet als plagiaat zou beschouwd worden, ook indien je twijfelt over het correct, toegelaten gebruik van AI-tools.

[2] Onderwijs- en Examenreglement